

Stream 异步执行、GPU 与 MPI 通信、双卡并行

目 录

1 引言：单卡也能并行，通信与计算重叠是关键	2
2 CUDA Stream 与异步执行	2
2.1 stream 级并行掩盖 launch 延迟	2
2.2 异步内存操作与计算重叠	2
2.3 分析 kernel 与主演化的错开	2
2.4 避免不必要的全局同步	3
2.5 正确性敏感：数据依赖与顺序保证	3
3 GPU 与 MPI 通信	3
3.1 MPI 进程数与 GPU 争用	3
3.2 MPI 初始化与不必要的等待	3
3.3 通信开销明显时的探索方向	4
4 双卡并行 (Bonus)	4
4.1 多卡运行方式与 rank 绑定	4
4.2 与单卡分开报告	4
4.3 评估 CUDA-aware MPI 与 NCCL	4
5 修改范围与限制	5
5.1 允许修改的范围	5
5.2 必须保证数学算法等价	5
5.3 严禁的修改	5
5.4 AMSS_NCKU_Input.py 的严格限制	5
6 正确性验证	5
7 性能评分	6
8 手算例子：用 stream 重叠 ghost exchange	6
9 本章你将学会	6
10 要点速查	7
11 小结	7

1 引言：单卡也能并行，通信与计算重叠是关键

上一章我们谈的是如何让单个 kernel 与访存更高效。但即便每个 kernel 都跑得很快，如果它们被串行发射、且每一步都做全局同步，GPU 还是会有大量时间在等待。本章我们把视线从 kernel 内部跳到 kernel 之间：用 **CUDA stream**（CUDA 流）把独立的操作放到不同流里，让 launch 延迟被掩盖、让通信与计算重叠。

这里有一个关键直觉：单卡程序并不等于“串行程序”。即使你只有一张 V100，只要程序里存在相互独立的操作（不同 patch 的演化、分析量计算、host 与 device 之间的数据搬运），就可以用多个 stream 把它们错开执行。这种错开不是把一个 kernel 拆成两个，而是让本可以并行的部分真正并行。

直觉 把单卡 GPU 想成一家有几条独立产线的车间。哪怕只有一栋厂房，只要几条产线的任务彼此独立，就可以让它们同时开工，而不是轮流用同一条产线。stream 就是给每条产线发一个独立工单的机制。

本章我们分四块：CUDA stream 与异步执行、GPU 与 MPI 通信的配合、双卡并行的 bonus 尝试、修改范围与正确性约束。最后用手算例子说明通信被 stream 重叠后能节省多少时间。

2 CUDA Stream 与异步执行

CUDA 的所有操作默认在 **default stream**（默认流）上执行，且默认流上的操作是串行的。如果你不显式创建 stream，所有 kernel launch、所有 memcpy 都排队执行，没有重叠空间。要实现并行，必须显式创建多个 stream，并把独立操作分配到不同 stream 上。

2.1 stream 级并行掩盖 launch 延迟

每次 kernel launch 在 host 侧有几十微秒的开销。如果一个时间步要发几十个 kernel，launch 开销累加就很可观。如果这些 kernel 之间相互独立，把它们放到不同 stream，host 就可以连续发起 launch 而不必等前一个 kernel 结束，launch 延迟被 GPU 上的实际执行掩盖。

2.2 异步内存操作与计算重叠

cudaMemcpyAsync 可以把一次拷贝放到指定 stream 上异步执行。在 ghost exchange 场景里，把 ghost 数据的 D2H 拷贝放到一个 stream，同时让计算 stream 继续做内部点的演化，就能让拷贝与计算重叠。这是单卡场景下 stream 最大的收益点。

2.3 分析 kernel 与主演化的错开

ABEGPU 在演化过程中要周期性计算分析量（例如表面积分、ADM 质量）。这些分析 kernel 与主演化在数据上有依赖（要读当前演化态），但在时间上不必紧跟在每个 RK 子步之后。如果分析 kernel 只在每隔若干步才需要输出，就可以放到独立 stream，让它在主演化推进下一步的同时跑前一步的分析。

分析 *kernel* 与演化 *kernel* 的数据依赖要非常清楚。分析读取的演化态必须是一个完整写好的时间步，不能读到半个 RK 子步的中间态。这种“读完整步”的保证通常用 *event* 来实现：演化写完一步后 *record* 一个 *event*，分析 *stream* 在 *read* 前 *wait* 这个 *event*。

2.4 避免不必要的全局同步

一个常见的低效模式是过早调用 `cudaDeviceSynchronize`。这个调用会让 host 阻塞等待 GPU 上所有 *stream* 全部完成。如果只是为了确保下一步能读到数据，本来只需要 *sync* 对应的 *stream*，却误用了 device 级同步，就把所有 *stream* 都拉下水串行了。

直觉 `device sync` 像是“全场暂停”，`stream sync` 像是“让某条产线停下来等齐”。如果只想确认一条产线完工，就别喊全场暂停，否则其他产线也被迫停下。

2.5 正确性敏感：数据依赖与顺序保证

stream 优化的核心难点不是写法，而是正确性。你必须清楚：每个数据依赖发生在哪两个操作之间，写操作在哪个 *stream*，读操作在哪个 *stream*。跨 *stream* 的依赖必须显式用 *event*（事件）或 *stream ordering* 来保证顺序，否则会出现“读到一半数据”的竞态。

1. **event**: 在写 *stream* 上 *record* 一个 *event*，在读 *stream* 上 *wait* 这个 *event*，保证读发生在写之后。
2. **stream ordering**: 用 `cudaStreamWaitEvent` 让一个 *stream* 等待另一个 *stream* 上的 *event*，建立跨流偏序。

经验：先把所有数据依赖画成一张图，标清楚每个依赖用哪个 *event* 连接，再去写代码。直接写代码再调试竞态，往往要花几倍时间。

3 GPU 与 MPI 通信

ABEGPU 的 GPU 模式下，程序仍然通过 MPI 启动。这意味着即使你只用一张卡，也要关心 MPI 配置。一个常见误区是“反正只用一卡，MPI 怎么配都无所谓”，其实不然。

3.1 MPI 进程数与 GPU 争用

如果 `MPI_processes` 设为大于 1，多个 *rank* 可能被绑定到同一张 GPU 上，导致它们争用 SM 与显存，反而比单 *rank* 慢。GPU 模式下通常应把 `MPI_processes` 设为 1 或与 GPU 数量匹配。要在 `AMSS_NCKU_Input.py` 里检查并调整。

有些同学误以为多开几个 *rank* 就能“用满 GPU”，其实多 *rank* 在同一张卡上会互相挤占，且 MPI 通信开销还在。除非你有明确的分卡策略，否则单卡用单 *rank* 更稳。

3.2 MPI 初始化与不必要的等待

MPI 初始化、*rank* 间通信、与 GPU *kernel* 之间可能存在不必要的等待。例如 host 在 *launch kernel* 后立刻 `MPI_Send`，但没有先 *sync* 对应 *stream*，就可能发出未写完的数据；反过

来，如果 host 在每次 launch 后都做一次 `cudaDeviceSynchronize` 再通信，又会把所有 stream 拉串行。正确的做法是用 event 或 stream sync 精确同步到通信需要的那一块数据。

3.3 通信开销明显时的探索方向

当 profiling 显示通信开销显著时，可以从这些方向探索：

1. **CUDA-aware 支持**：确认你用的 MPI 实现是否被构建为支持 CUDA-aware buffer。默认构建通常未启用，需要修改 `macrodef.h` 里的 `MPI_CUDA_AWARE` 宏。
2. **device 直传 vs host staging**：启用 CUDA-aware 后，比较 device buffer 直接通信与默认 host staging 的端到端时间。
3. **合并小消息**：ghost exchange 如果每次发很多小消息，可以尝试把同一 patch 的多个边界合并成一个大消息，减少高频发送的固定开销。
4. **发送/接收顺序**：调整 send 与 recv 的顺序，让一侧的 send 与另一侧的 recv 在时间上对齐，减少等待。
5. **通信与独立 kernel 重叠**：把通信放到独立 stream，与不依赖该通信结果的 kernel 重叠执行。

直觉 把 MPI 通信想成寄快递。CUDA-aware 是“快递员直接上门取货台”，host staging 是“先搬到前台再让快递员取”。后者多一次搬运但兼容所有快递公司，前者省一次搬运但要求快递公司支持。合并小消息像把多个包裹装一个箱子，省的是每个包裹的运单费。

4 双卡并行 (Bonus)

Lab4 的基线是单卡 V100，bonus 部分允许尝试 2 卡。双卡并行的核心问题是：怎么把一个演化问题切到两张卡上，让它们各自算一部分，再在边界做通信。

4.1 多卡运行方式与 rank 绑定

双卡通常仍以 MPI 启动，两个 rank 各绑定一张 GPU。需要明确：

1. **运行方式**：用 `mpiexec -n 2` 启动，每个 rank 用 `cudaSetDevice` 绑定到不同 GPU。
2. **rank/GPU 绑定**：通常用 `local_rank % 2` 或环境变量 `CUDA_VISIBLE_DEVICES` 来分配。
3. **通信库**：rank 间通信走 MPI，可选 CUDA-aware MPI 或 NCCL。
4. **数据路径**：每张卡持有半个网格，边界数据要通过通信库在卡间搬运。

4.2 与单卡分开报告

bonus 评分不设单独性能曲线，只展示双卡 vs 单卡的提升。这意味着双卡报告要同时给出单卡基线时间与双卡时间，并说明加速比。不要把双卡结果混进单卡曲线里。

4.3 评估 CUDA-aware MPI 与 NCCL

对 GPU 间通信，有两种主流方案：

方案	优点	缺点
CUDA-aware MPI	与现有 MPI 代码兼容，改动小	带宽受限于 MPI 实现，不一定最优

方案	优点	缺点
NCCL (NVIDIA Collective Communications Library)	针对 GPU 通信优化, 带宽高	需要重写通信代码为集合通信

NCCL 适合做集合通信 (*allreduce*、*allgather* 等), 对点到点 *ghost exchange* 不一定比 *CUDA-aware MPI* 快。要不要换 NCCL 要看你通信模式是集合式还是点到点。

5 修改范围与限制

bonus 与优化都有一套明确的修改边界。违反边界即便性能提升, 也不会被认可。

5.1 允许修改的范围

1. `src/lab4/src/` 下的 C++、Fortran、CUDA 源代码。
2. `CMakeLists.txt`、`compile.sh`、`run.sh` 及辅助文件。

5.2 必须保证数学算法等价

所有修改必须保证数学算法等价性。也就是说, 你可以改变实现方式 (kernel 拆分、stream 调度、通信路径), 但不能改变差分格式、耗散阶数、RK 方案、AMR 规则等数学内容。

5.3 严禁的修改

1. 减少物理计算量。
2. 降低网格规模。
3. 缩短演化时间。
4. 跳过输出。
5. 读取预计算答案。
6. 修改评测输入。
7. 把关键计算替换为低精度 (除非你解释清楚如何保持高精度并仍然通过正确性)。

5.4 AMSS_NCKU_Input.py 的严格限制

`AMSS_NCKU_Input.py` 在正式评测中只允许修改 MPI、OpenMP、GPU 相关参数。物理参数、网格规模、演化时间、输出间隔严禁修改。调试时可以临时缩短 `Final_Evolution_Time` 加快迭代, 但提交时必须恢复, 且缩短时间的结果不能作为评分依据。

直觉 把这些限制想成“赛道规则”。你可以改装引擎 (kernel 优化)、调整换挡策略 (stream 调度)、换轮胎 (通信库), 但不能缩短赛道 (演化时间)、减少圈数 (物理计算)、抄近路 (读预计算答案)。违规的圈速不算成绩。

6 正确性验证

所有优化必须通过正确性校验。Lab4 的校验标准是:

1. bssn_BH.dat: 六坐标列的相对 **RMS** (均方根) 误差 $< 0.1\%$ 。
2. bssn_constraint.dat: Grid Level 0 的 **Hamiltonian constraint** (哈密顿约束) 与 **momentum constraint** (动量约束) 的绝对值最大值 ≤ 2.0 。

这两条是物理意义上的“可信阈值”。约束量是 BSSN 方程的残差，理论上应为零，离散后会有数值误差，绝对值 ≤ 2.0 是可接受范围。RMS 误差 $< 0.1\%$ 保证你的轨道与参考解一致。

调试阶段可以缩短 Final_Evolution_Time，但提交必须恢复原值。缩短时间的结果只用来快速验证逻辑正确性，不能作为性能评分依据。

7 性能评分

性能评分基于端到端时间。正式计时不会使用 **TwoPuncture** (两奇点初值求解器) 的缓存，也就是说初值求解的时间也算进总时间。这意味着如果你只优化了演化部分而忽略初值求解，端到端收益会被稀释。

直觉 正式计时不用 TwoPuncture 缓存，是为了防止“靠缓存白嫖初值时间”的取巧。这也提醒你：优化要看整程序的端到端，而不是只盯演化热点。

8 手算例子：用 stream 重叠 ghost exchange

例 设单卡演化一步的总时间为 100 ms，其中 ghost exchange (含 D2H、MPI 通信、H2D) 占 20 ms，纯计算占 80 ms。如果 ghost exchange 与下一步的内部计算在数据上独立 (即 ghost 只影响边界点，而内部点可以先算)，就可以把 exchange 放到一个独立 stream，让它在内部计算 stream 跑的同时执行。假设 exchange 被掩盖到只剩 5 ms 的不可重叠部分 (例如最后的边界同步)，那么一步时间从 $80 + 20 = 100$ ms 降到 $80 + 5 = 85$ ms，加速比 $\frac{100}{85} \approx 1.18 \times$ 。演化 1000 步共节省 $15 \times 1000 = 15000$ ms = 15 s。若演化步数更多或 exchange 占比更高，收益还会放大。这个例子说明：stream 重叠的收益取决于“可被掩盖的通信比例”与“不可被掩盖的同步尾巴”。

注意这里有两个关键假设：第一，ghost exchange 与内部计算在数据上独立 (边界点之外的区域可以先算)；第二，exchange 内部还有 5 ms 不可被掩盖 (例如接收端必须等数据到齐才能更新边界)。如果第一个假设不成立 (例如边界立即影响内部)，重叠空间就小得多。

9 本章你将学会

1. 用 CUDA stream 重叠计算与通信，掩盖 launch 延迟，避免不必要的全局同步。
2. 配置 GPU 模式下的 MPI 参数，避免多 rank 争用同一张卡。
3. 探索 CUDA-aware MPI 与消息合并、顺序调整等通信优化。
4. 尝试双卡 bonus，说明 rank/GPU 绑定、通信库、数据路径，并与单卡分开报告。
5. 遵守修改范围与正确性约束，保证数学等价、不缩短演化时间、通过约束与 RMS 校验。

10 要点速查

主题	要点
stream 并行	独立操作放不同 stream, 掩盖 launch 延迟, 通信与计算重叠
异步内存	cudaMemcpyAsync 放 stream, ghost 拷贝与计算重叠
跨流依赖	用 event 与 cudaStreamWaitEvent 保证顺序
避免全局同步	不要过早 cudaDeviceSynchronize, 用 stream sync 精确同步
MPI 进程数	GPU 模式通常 MPI_processes = 1 或与 GPU 数匹配
CUDA-aware	MPI_CUDA_AWARE 默认 0, 启用后比较 device 直传 vs host staging
通信优化	合并小消息、调整 send/recv 顺序、与独立 kernel 重叠
双卡 bonus	rank/GPU 绑定, CUDA-aware MPI 或 NCCL, 与单卡分开报告
修改范围	src/lab4/src/, CMake、compile.sh、run.sh, 保证数学等价
严禁项	减物理计算、降网格、缩短时间、跳输出、读预计算、改评测输入
Input 限制	仅 MPI/OpenMP/GPU 参数可改, 物理/网格/时间/输出间隔严禁
正确性	BH 6 列 $RMS < 0.1\%$, Level 0 约束 $abs \leq 2.0$
性能评分	端到端时间, 正式计时不用 TwoPuncture 缓存

11 小结

本章我们跳出单 kernel 视角, 看 kernel 之间如何通过 CUDA stream 实现异步与重叠。核心是把独立的操作 (不同 patch、分析量、数据搬运) 放到不同 stream, 用 event 精确保证跨流依赖, 避免不必要的全局同步。在 GPU 与 MPI 配合上, 要正确设置 MPI 进程数, 避免多 rank 争用同一卡, 并在通信开销明显时探索 CUDA-aware、消息合并与顺序调整。双卡 bonus 需要说明 rank 绑定、通信库与数据路径, 并与单卡分开报告。所有优化都必须在修改范围内、保证数学等价、通过 RMS 与约束正确性校验。下一章我们会把这些手段综合起来, 讨论如何把单卡 ABEGPU 端到端跑出更短时间。

讲义基于 HPC101 2025 Lab4 实验内容编写